

PLSQL Exercise Output

Database Schema Used:

The banking database system is created using multiple related tables to store and manage customer, account, transaction, loan, and employee information. The following tables are used in the PL/SQL programs:

- Customers – Stores customer details such as customer ID, name, date of birth, balance, and VIP status.
- Accounts – Stores account details including account type and account balance linked with customers.
- Transactions – Stores deposit and withdrawal transaction records for customer accounts.
- Loans – Stores loan details including loan amount, interest rate, and loan duration.
- Employees – Stores employee details such as name, position, department, and salary.
- ErrorLogs – Stores error messages generated during exception handling.

Primary keys and foreign keys are used to establish relationships between tables and maintain data consistency. Sample records are inserted into each table to test the PL/SQL programs.

Output:

Customer Table:

Query result Script output DBMS output Explain Plan SQL history						
Download Execution time: 0.002 seconds						
	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	5/15/1963, 12:00:00	1000	6/22/2026, 12:57:41	N
2	2	Jane Smith	7/20/1990, 12:00:00	1500	6/22/2026, 12:57:41	N

Accounts table:

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1		1	1 Savings	540	6/22/2026, 12:57:41
2		2	2 Checking	2000	6/22/2026, 12:57:41

Transactions table:

Query result					
Script output					
DBMS output					
Explain Plan					
SQL history					
Download Execution time: 0.006 seconds					
	TRANSACTIONID	ACCOUNTID	TRANSACTIONDATE	AMOUNT	TRANSACTIONTYPE
1	1	1	6/22/2026, 12:57:4!	200	Deposit
2	2	2	6/22/2026, 12:57:4!	300	Withdrawal

Loans table:

Query result						
Script output						
DBMS output						
Explain Plan						
SQL history						
Download Execution time: 0.005 seconds						
	LOANID	CUSTOMERID	LOANAMOUNT	INTERESTRATE	STARTDATE	ENDDATE
1	1	1	5000	5	6/22/2026, 12:57:4!	6/22/2031, 12:57:4!
2	2	2	10000	5	6/22/2026, 12:57:4!	7/17/2026, 12:57:4!

Employees table:

Query result							
Script output							
DBMS output							
Explain Plan							
SQL history							
Download Execution time: 0.006 seconds							
	EMPLOYEEID	NAME	POSITION	SALARY	DEPARTMENT	DEPARTMENTID	HIREDATE
1	1	Alice Johnson	Manager	70000	HR	(null)	6/15/2015, 12:00:00
2	2	Bob Brown	Developer	60000	IT	(null)	3/20/2017, 12:00:00

The database tables are created successfully and sample values are inserted into all tables. The output displays customer, account, transaction, loan, and employee details.

Exercise 1: Control Structures in PL/SQL

Scenario :

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  ↻  ⌵  Aa  🗑️

1 BEGIN
2     DEMS_OUTPUT.PUT_LINE('SCENARIO - 1: Interest Discount');
3     ApplyInterestDiscount;
4 END;
5 /
```

Output :

```
SCENARIO - 1: Interest Discount
CustomerID: 1 interest rate is decreased by 1%.

SCENARIO - 1: Interest Discount
CustomerID: 1 interest rate is decreased by 1%.
```

Scenario 2: A customer can be promoted to VIP status based on their balance.

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  ↻  ⌵  Aa  🗑️

1 BEGIN
2     DEMS_OUTPUT.PUT_LINE('SCENARIO - 2: VIP Status Update');
3     PromoteToVIP;
4 END;
5 /
6 SELECT CustomerID, Name, Balance, IsVIP
7 FROM Customers;
```

Output:

SCENARIO - 2: VIP Status Update
CustomerID: 1 demoted from VIP.
CustomerID: 2 demoted from VIP.

Query result Script output DBMS output Explain Plan SQL history					
  Download Execution time: 0.006 seconds					
	CUSTOMERID	NAME	BALANCE	ISVIP	
1	1	John Doe	1000	N	
2	2	Jane Smith	1500	N	

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
[ SQL Worksheet ]*    ▶    ≡    🔗    ↺    ≡    Aa    ▼    🗑️
```

```
1 BEGIN
2     DBMS_OUTPUT.PUT_LINE('SCENARIO - 3: Loan Reminder');
3     SendLoanReminders;
4 END;
5 /
```

Output:

SCENARIO - 3: Loan Reminder
Reminder: Loan due on 2026-07-17 for CustomerID: 2

Exercise 3: Stored Procedures in PL/SQL

Scenario :

Scenario 1: The bank needs to process monthly interest for all savings accounts.

Question: Write a stored procedure `ProcessMonthlyInterest` that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
[ SQL Worksheet ]*  ▶  ⌵  🔗  📄  ⌵  Aa  🗑️

1  BEGIN
2      ProcessMonthlyInterest;
3  END;
4  /
5  SELECT AccountID, AccountType, Balance
6  FROM Accounts;
```

Output :

Query result Script output DBMS output Explain Plan SQL history				
🗑️	📄	Download	Execution time: 0.005 seconds	
	ACCOUNTID	ACCOUNTTYPE	BALANCE	
1	1	Savings	540	
2	2	Checking	2000	

Number of accounts updated: 1

Query result Script output DBMS output Explain Plan SQL history				
🗑️	📄	Download	Execution time: 0.005 seconds	
	ACCOUNTID	ACCOUNTTYPE	BALANCE	
1	1	Savings	545	
2	2	Checking	2000	

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

Question: Write a stored procedure `UpdateEmployeeBonus` that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
[ SQL Worksheet ]*  ▶  ⌵  🔗  📄  ⌵  Aa  🗑️

1  BEGIN
2      UpdateEmployeeBonus (1,10);
3  END;
4  /
5
6  SELECT EmployeeID, Name, Salary
7  FROM Employees;
```

Output:

Query result				Script output	DBMS output	Explain Plan	SQL history
				Download Execution time: 0.005 seconds			
	EMPLOYEEID	NAME		SALARY			
1	1	Alice Johnson		70000			
2	2	Bob Brown		60000			

Scenario 3: Customers should be able to transfer funds between their accounts.

Question: Write a stored procedure `TransferFunds` that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
[ SQL Worksheet ]*  ▶  ⌵  🔑  🔄  ⌵  Aa  🗑️

1  SELECT AccountID, Balance
2  FROM Accounts;
3
4
5  BEGIN
6      TransferFunds (1,2,100);
7  END;
8  /
9
10
11 SELECT AccountID, Balance
12 FROM Accounts;
```

Output:

Before Transfer

🗑️ ⓘ Download ⌵ Execution time: 0.005 seconds		
	ACCOUNTID	BALANCE
1	1	545
2	2	2000

After Transfer:

Query result Script output DBMS output Explain Plan SQL history				
🗑️ ⓘ Download ⌵ Execution time: 0.002 seconds				
	ACCOUNTID	BALANCE		
1	1	445		
2	2	2100		